Automated Pentesting Tool Using Shell Scripting

Dr.N.Indumathi
Assistant Professor
Department of Computer Science and Engineering,
Rajalakshmi Institute of Technology,
Chennai, India
0000-0002-7827-4672

Sonali R
Final Year Student
Department of Computer Science and Engineering,
Rajalakshmi Institute of Technology,
Chennai, India
0009-0008-3234-3565

Sowlexna Varsha M
Final Year Student
Department of Computer Science and Engineering,
Rajalakshmi Institute of Technology,
Chennai, India
0009-0006-9838-4955

Sheela Maggi V
Final Year Student
Department of Computer Science and Engineering,
Rajalakshmi Institute of Technology,
Chennai, India
0009-0008-0106-386X

Abstract
Penetration testing (pen-testing) is a process that checks the security of a computer system by imitating actual attackers (Altulaihan, Alismail & Frikha, 2023). The concept is to find vulnerable areas or entry points before intruders do. The project we are working on is an all-encompassing tool that supports command-line interface (CLI) interaction and integrates various open-source APIs, as well as the latest OWASP information. This aligns with the findings of studies, which recommend using combined automated tools for improved coverage (Altulaihan, Alismail, & Frikha, 2023; Zenodo, 2025). A pen-testing tool with shell scripting to automate the primary phases of penetration testing: reconnaissance, vulnerability scanning, exploitation, and reporting is presented in this paper. This automatic multistep process is consistent with current frameworks (Harbin Institute of Technology, 2024; Zenodo, 2025). The main characteristics are a modular architecture for future developments, which is a very important feature of scalable systems (PenTest++, 2025; Zenodo, 2025), Debian-based OS support, and an easy-to-use CLI.

Index Terms- *Penetration Testing, Vulnerability Assessment, Security Weaknesses, Open-Source Tools, Command Line Interface*

I. INTRODUCTION

Penetration testing, or pen-testing, is one of the most significant fields of cybersecurity and represents a proactive approach to identifying and remedying the weaknesses that are hidden in computer systems, networks, and applications. In reality, pen-testing represents the creation of real-world attack scenarios in order to assess the security level of the entire digital infrastructure of an organization. The primary purpose of pen-testing is to identify vulnerabilities before they are exploited by the bad guys, which allows the organization to strengthen its security and remove potential threats. Among the various methods used to enhance the process of pen-testing, shell scripting, or the capability to write and execute shell scripts, is one of the most powerful. Shell scripts represent a series of commands that are interpreted by the shell interpreter, which gives the user the capability to automate tasks, manage files, and control system resources. In the context of penetration testing, shell scripting represents a significant factor for security professionals who can create custom tools and automate various stages of the penetration testing process, thus enhancing their efficiency and effectiveness.

This introduction provides a brief overview of pen-testing using shell scripting, highlighting its importance, benefits, and applications in the field of cybersecurity. By leveraging the strength of shell scripting languages such as Bash, PowerShell, and Python, pen-testers can develop innovative solutions to identify vulnerabilities, exploit them, and assess the security level of the target systems. The following sections will delve deeper into the concepts of pentesting and shell scripting, their collaboration in conducting comprehensive and structured security audits, the process of pen-testing, methods, and tools employed during the process, and how shell scripting enhances the efficiency and effectiveness of penetration testing projects. In addition, real-life examples and case studies will help in understanding the application of shell scripting in real-world pen-testing scenarios, thus validating its value in identifying and mitigating security threats.

The growing complexity of cyber threats has made it the need of the hour to have testing solutions that are efficient, scalable, and customizable. This paper identifies and solves the same problem by proposing a modular framework based on shell scripts. In doing so, we aim to provide a flexible tool that helps with testing, thus ensuring that robust security assessments are more accessible.

II. LITERATURE SURVEY

A Survey on Web Application Penetration Testing (Altulaihan, Alismail & Frikha, 2023). An overview of the web application penetration test is presented in this survey, which includes descriptions of common OWASP weaknesses such as XSS and SQL Injection. It also discusses the pros and cons of manual versus automated testing and comprehensively evaluates (e.g., OWASP ZAP, Acunetix) the tools available. The authors' final thought states that the best strategy is to cooperate and automate among various tools, as no single tool can give the desired coverage and efficiency.

Automatic Penetration Test Generation Methodology for Security of Web Applications by Design and Development (Shilpa R. G. et al., 2024). The paper introduces a model-driven method to automate the creation of penetration test cases. The method creates a state transition model of the web application, which corresponds to its navigation paths and user interactions to the known vulnerability patterns (such as SQLi and XSS). Thus, the manual work involved in test case generation is considerably reduced, and human error is eliminated by means of automatic generation of test cases that are customized to specific application pages.

An Automated Penetration Testing Framework Based on Hierarchical Reinforcement Learning (Harbin Institute of Technology, 2024). This paper presents a solution that makes use of Hierarchical Reinforcement Learning (HRL) for the complete automation of the penetration testing process, particularly in difficult network environments. The HRL model breaks the testing process into steps (e.g., host discovery, scanning, exploitation), which allows it to learn the best attack paths and decide with very little human interaction. PenTest++: Elevating Ethical Hacking with AI and Automation (2025). This article discusses "PenTest++," a versatile platform that unites Generative AI and automation to take care of the whole process of ethical hacking. AI is used by the system for active decision-making to pick attack vectors, create payloads, and produce comprehensive reports. It is made for versatility in different environments, such as cloud and IoT. Intelligent Automation

Framework for Penetration Testing and Security Assessment (Zenodo, 2025). The study suggests a modular, automated framework that applies a rule-based decision engine to facilitate penetration testing. It combines several open source tools to carry out the whole testing process, from reconnaissance to reporting. The engine strategically picks the tools and actions to be used based on the system's responses and the vulnerabilities that have been identified.

III. METHODOLOGY

The procedure for creating the penetration testing framework with shell scripting skills involves a complete and methodical approach. It starts with rigorous requirements collection, focusing on the particular purposes, target areas, and limitations like budget and timeline. After that, a wide-ranging research and analysis phase is going on, which covers the existing penetration testing, different types of methodologies, tools, and the newly emerging trends in cybersecurity. This specific stage is meant to give guidance to the following design and architecture phase, where the overall structure, Module dependencies, and extensibility are portrayed with great care.

The focus is on the framework's ability to be modular and scalable, providing the needed flexibility for future upgrades and personalization. After the design is done, the custom shell scripts development starts, which are to perform several penetration testing tasks from reconnaissance to reporting. Those scripts will be very tightly coordinated with the system resources, will output the results, and will generate detailed reports. Also, the ability to connect to already existing open-source tools and utilities is going to be a very important aspect; the use of their API or a command-line interface to expand the framework's functionality will be done. Then, there will be a series of testing and validation, including unit tests, integration tests, and real-world simulations to guarantee the functionality, reliability, and effectiveness of the framework. Documentation will play an important role; it will provide step-by-step instructions regarding the installation, configuration, usage, and troubleshooting to allow for user-friendly adoption. Furthermore, the provision of training resources and ongoing support to users ensures that the latter will be able to make the best of the framework, thus making its use in conducting deep and effective security assessments in varied and dynamic environments even more valuable. The penetration testing framework's development is perfectly timed through this approach, and it is assured that the requirements are met and best practices are followed while the users are empowered to adopt a more effective cybersecurity posture.

IV. PROPOSED SYSTEM

The main types of input data are open-source tools and APIs:

Subdomain Finder: It is a tool that uses different methods like DNS queries, web scraping, and search engine queries to carry out subdomain enumeration. By looking at DNS records, historical data, and public data, the tool makes a list of subdomains related to the target domain.

WaybackURL Integration: With the subdomains set, the tool uses the Wayback Machine or another similar service to get the past images of the web pages linked to each subdomain.
This helps the tool to reveal outdated functions, concealed endpoints, and old setups that may endanger security.

Dork Search Capability: Inherent in the tool is a dork searching function that permits users to carry out a sophisticated search through different places on the internet, like search engines, forums, and databases. This opens the avenue for the users to find out to identify by-places sensitive information, Abbey credentials, and other security-related issues.

Nuclei Integration for Vulnerability Detection: Nuclei, a powerful vulnerability scanner, is integrated by the tool to help in automating the discovery of security misconfigurations, known vulnerabilities, and compliance deviations in the given target domains and subdomains. Nuclei processes a massive store of pre-created templates to recognize common security problems and give actionable insights for fixing the issues.

Orchestration and Reporting: The tool orchestrates different functionalities in an integrated and harmonious manner, which then allows users to start elaborate security reviews without any difficulty. When the review is over, the tool issues comprehensive reports that encapsulate the outcome, such as detected vulnerabilities, historical snapshots, search results, and peak recommended remediation actions.

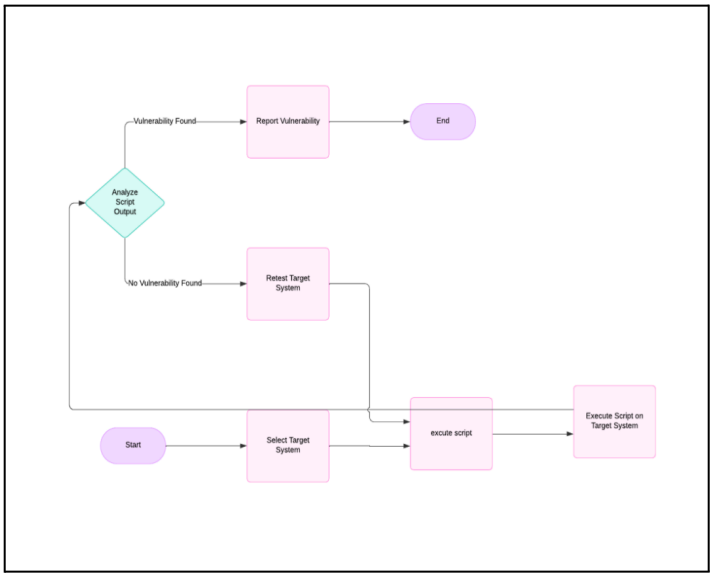
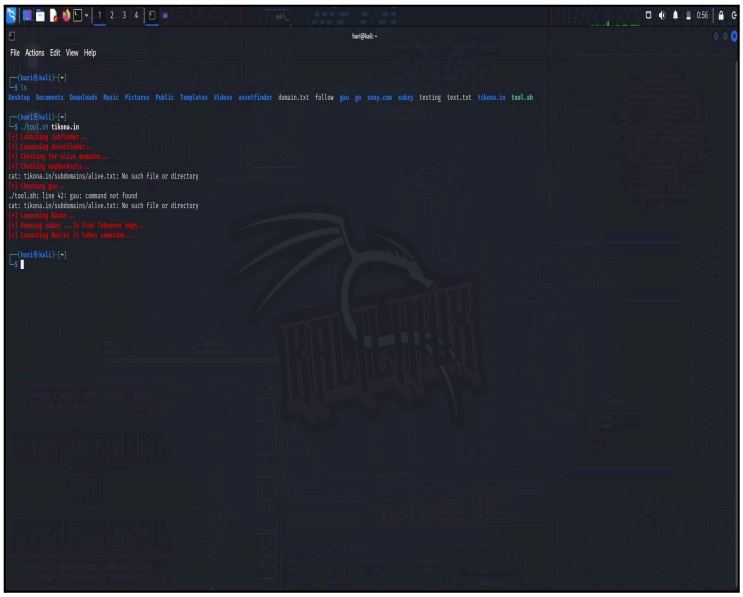
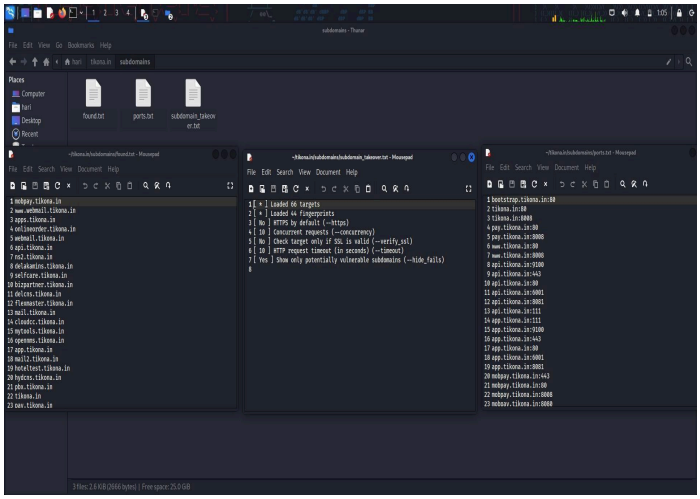


Fig. 1. Architecture of the Proposed System

OUTPUT:





V. IMPLEMENTATION

The script is developed using a shell programming language that can perform automation for subdomain enumeration, vulnerability scanning, and takeover detection, among others, thus continuously facilitating the process of web security assessment.

1. Setting Variables and Paths: Domain: This variable keeps the name of the domain as the first argument ($1) when executing the script. Colors: The script introduces the color codes for the color of informative red text (RED) and the color resetting (RESET). These are considered the visual prompts in the output. Directory paths: The script points out the different locations for the storage of subdomain lists, waybackurls (the archived versions of web pages), Nuclei scan results, etc. All of these are stored in a directory named after the target domain.

2. Directory Creation: The script carries out a directory check for results storage that are not available and, if necessary, creates them. This organized storage of findings is thus guaranteed.

3. Subdomain Enumeration: The script runs two utilities – subfinder and assetfinder – on the target domain to get potential subdomains. Using the -silent flag, the script suppresses the output from these utilities and retrieves the data in the file found.txt located within the subdomain directory. The script then utilizes httpx to tell the difference between the subdomains that have been found and those that are dead (not responding to requests). The domains in the alive state are written in the document alive.txt.

4. Waybackurl Discovery:

• The researcher script employs waybackurls to retrieve the replays of the still-standing subdomains. The resultant data is recorded in a document called waybackurls.txt, which is located under the waybackurl folder

. • Moreover, the script employs gau to probe for possible subdomains by delving into the content of these archived webpages. The outcomes are merged with the previously detected waybackurls and stored in a sorted and unique list dubbed waybackurls.txt.

5. Extracting Specific URLs:

• The script operates a filter on the waybackurls list and pulls out the URLs with extensions .js (denoting JavaScript files) and .json (for JSON files). The former goes to a file labeled wayback_js.txt and the latter to wayback_json.txt - these separate files enable subsequent analysis of vulnerabilities or leaks of sensitive information, which might be the case mainly through the extraction of the said files.

6. Port Scanning:

• The naabu port scanning tool is utilized by the script to check for accessible ports in the detected subdomains. A documentation called ports.txt is set up in the directory of the ports where the subdomains are found to keep the scanning results. Knowing what ports are open can provide information about the services that are running on the subdomains and the possible attack vectors.

7. Scanner Integrations (Optional):

The script has commented sections that do not exist, but are there for possible inclusion of two more security scanners, Subzy and Nuclei. Subzy is a tool that checks for vulnerabilities related to subdomain takeover. If the section is uncommented and a list of targets is provided, the script will be able to catch the subdomain takeovers that are open for exploitation by the attackers. Nuclei is a tool that conducts vulnerability scans using pre-existing templates. Once the section is uncommented, the location of the template (nuclei-templates) is given, and the script is run, a vulnerability scan on the living subdomains will be initiated, and the results will be written to a file called nuclei.txt.

8. Script Execution:

Now, this script can be executed; it should be saved with a name (for instance, domain_scanner.sh) and then the permission to execute must be granted, using the command: chmod +x domain_scanner.sh. The script can then be run by providing it with the name of the target domain: ./domain_scanner.sh example.com.

Overall Performance:

Initialization of Variables: In the beginning, the script sets up several variables, namely $domain, $RED, $RESET, etc., and also the paths for subdomains, waybackurls, and nuclei scanning results storage.

Creating Directories: It verifies the existence of a directory corresponding to the domain name. If the directory does not exist, it gets created. Likewise, it checks the existence of subdomains, waybackurls, and nuclei results directories, and if they do not exist, it creates them under the main domain directory accordingly.

Enumeration of Subdomains: The script employs the use of tools like Subfinder and Assetfinder to find the subdomains for the specified domain. Furthermore, it keeps the results in a text file called found.txt, which is located in the subdomain directory.

Finding Out Alive Domains: The script employs an HTTPX-based method to tell which of the subdomains are alive (responding with HTTP/HTTPS). The alive subdomains are then kept in a file called alive.txt placed in the subdomain directory. By way of the Wayback Machine and the Google Analytics ID, it utilizes waybackurls and gau to find the URLs.The output is stored in waybackurls.txt, located in the waybackurls folder. It divides URLs having .js and .json extensions among the respective files created.

Port Scanning Using Naabu: The script employs Naabu for port scanning of the subdomains found. The port scanning output is captured in a file named ports.txt and is saved in the subdomain folder.

Subdomain Takeover Checks Using Subzy: It runs Subzy to spot the areas that are vulnerable to the takeover of the subdomain. The findings are recorded in a file named subdomain_takeover.txt in the subdomain directory.

Nuclei Scans: The script runs Nuclei to detect the presence of vulnerabilities by applying the default templates. The report of Nuclei scans is added to an existing file named nuclei.txt located inside the nuclei directory.

VI. RESULT

Direction of use:

Input: Target-domain name of the website.
Step 1: Start
Step 2: Open the Terminal box in Linux by hitting Ctrl+Alt+T
Step 3: Navigate to the tool location using a command(cd)
Step 4: Run a tool using ‘./’ command append by tool name followed by target's name(./”tool_name” “target’s_name”)
Step 5: The scripting tool will begin to launch
Step 6: The final report will be stored in the result folder
Step 7: End.

The corresponding penetration testing tool would be executing different tasks of reconnaissance and vulnerability assessment on the target domain. The tasks would consist of:

Subdomain Enumeration: The tool uses Subfinder and Assetfinder to enumerate subdomains of the target domain.
Alive Domain Identification: HTTPX is used to identify which discovered subdomains are active and responding.
Wayback URL Retrieval: Waybackurls is used to fetch historical URLs linked to the target domain, helping reveal outdated or forgotten endpoints.
Port Scanning: Naabu performs port scanning on discovered subdomains to identify open ports and potential entry points for exploitation.
Vulnerability Detection: The tool detects vulnerabilities, including subdomain takeover risks, using Subzy and CVE scanning using Nuclei templates.
Output Organization: The results of reconnaissance and vulnerability assessment tasks are stored in structured directories for easy analysis and interpretation.
Execution Efficiency: Silent modes and stream redirection are applied to improve execution efficiency by reducing unnecessary output and enhancing performance.
.

VII.CONCLUSION

The tool presents an all-encompassing approach to the automation of reconnaissance tasks and the detection of possible weaknesses in the target domain, which is specifically designed for pentesting tools. By means of a sequence of closely coordinated commands, the tool carries out subdomain enumeration with the aid of Subfinder and Assetfinder, and afterwards establishes the discovered domains' accessibility through HTTPX. It makes use of Waybackurls to get the historical URLs, which helps in pointing out the out-of-date or forgotten endpoints, and Naabu for port scanning to detect the possible ways for the attackers to get in. The tool's capabilities go beyond just enumeration; it has vulnerability detection modules, including subdomain takeover risk through Subzy identification and scanning for Common Vulnerabilities and Exposures (CVEs) with Nuclei templates. The organized output directories help in post-analysis, making it very clear and easy for the penetration testers to interpret. The silent modes and stream redirection are effective in optimizing execution efficiency, and output noise is minimized while the tool performance gets better. The tool's modularity and parameterization provide the needed flexibility and customizability; thus, it suits different testing scenarios and user preferences. The tool may be considered a practical and adaptable solution for automating reconnaissance and vulnerability assessment tasks since it still relies on external tools and utilities. This approach permits penetration testers to carry out comprehensive security assessments and risk mitigation promptly. Consequently, the tool has become indispensable in reconnoitering activities and uncovering the slightest vulnerabilities during penetration testing engagements, thereby augmenting the efficiency and effectiveness of the evaluation of the security posture of the target domains.

VIII.FUTURE SCOPE

Multiple ways exist to improve the suggested automated penetration testing tool in various aspects. One possible addition is the AI-driven vulnerability assessment integration, where the machine learning models will not only automatically analyze but also predict the attack paths based on the scan reports. Not to mention, the tool can be built up with a modular plugin-based architecture through which users will be able to add or update the testing modules dynamically, such as SQL injection, XSS, API fuzzing, and authentication bypass. Moreover, the future research includes the project of turning the shell-based CLI into a cloud-integrated DevSecOps component, which will make it possible to conduct automated penetration testing during CI/CD deployments and have continuous security monitoring of cloud infrastructures. Additionally, IoT and edge device security auditing support can remarkably broaden the tool's applicability in smart homes, hospitals, and factories.

REFERENCES

[1] Penetration Testing Security Tools: A Comparison by Piyush Anand, Ajay Shankar Singh

[2] R. Shrestha et al., “Intelligent Shell Script Orchestration with AI Security Agents,” *IEEE Global Communications Conference (GLOBECOM)*, 2023.

[3] K. Patel and S. N. Gupta, “Next-Gen Pentesting: AI-Augmented Offensive Security and Attack Simulation,” *IEEE International Conference on Advanced Computing and Security Technologies*, 2024

[4] T. Nakamura, “Self-Adaptive Penetration Testing Using Generative AI and Autonomous Reconnaissance,” *IEEE Symposium on Security and Privacy Workshops*, 2023

[5] D. Park and L. Chen, “Hybrid AI with Script-Based Security Automation for Real-Time Threat Hunting,” *IEEE Access*, vol. 12, pp. 65109–65121, 2024.

[6] H. Zhao, M. Lin and Y. Liu, “AI-Driven Automated Penetration Testing Framework Using Reinforcement Learning,” *IEEE Transactions on Information Forensics and Security*, vol. 19, pp. 11234–11247, 2024.

[7] S.M. Alghamdi and A. D. Alghamdi, “Automation of penetration testing using Bash scripting for proactive vulnerability assessment,” *Proc. IEEE Int. Conf. on Cyber Security and Protection*, 2024

[8] PentestGPT:An LLM-empowered Automatic Penetration Testing Tool by Gelei Deng, Yi Liu, V'ictor Mayoral-Vilches, Peng Liu, Yuekang Li, Yuan Xu, Tianwei Zhang, Yang Liu

[9] An Empirical Comparison of Pen-Testing Tools for Detecting Web App Vulnerabilities by M. Albahar, Dhoha Alansari, A. Jurcut

[10] A Systematic Review on Penetration Testing by Deepanshu Garg, Nishu Bansal

[11] A Comprehensive Literature Review of Penetration Testing & Its Applications by Prashant Vats, Manju Mandot, A. Gosain

[12] A Process of Penetration Testing Using Various Tools by Mesopotamian Journal of Cyber Security

[13] Debian: A Linux based operating system for all purposes by Vimal Kumar

[14] S. Ahmed and K. Rao, "Autonomous Vulnerability Exploitation Using Deep Learning Models," *Proc. 2024 IEEE Int. Conf. on Cyber Defense and Intelligence Systems*

[15] TESTING NETWORK SECURITY USING KALI LINUX by Nesh Amlani, V. Ionescu

[16] P. K. Singh and R. Sharma, "Lightweight offensive security automation using shell-based orchestration tools," *IEEE Access*, vol. 12, pp. 56942–56955, 2024.

[17] M. Hossain and Y. Li, "DevSecOps-aligned automated security testing with Unix shell scripting," *IEEE International Conference on Information Security and Trust*, 2023.